



Thread e Service

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Il Main Thread (UI Thread)

Ogni app Android ha un singolo thread principale che gestisce il rendering e gli eventi dell'interfaccia utente.

- Disegna i frame dell'interfaccia (16 ms per frame = 60 fps).
- Riceve i tocchi dell'utente e distribuisce gli eventi.
- Esegue i callback del ciclo di vita (onCreate, onResume, ...).
- Se un'operazione dura > 5 secondi → ANR (Application Not Responding).
- Le operazioni di rete sul main thread lanciano `NetworkOnMainThreadException`.
- Regola d'oro: mai bloccare il main thread con I/O, calcoli pesanti o sleep.



ANR – Application Not Responding

Se il main thread è bloccato per più di 5 s, Android mostra un dialog di errore. L'utente può chiudere l'app.

Cosa succede quando blocchiamo la UI

Un'operazione lunga sul main thread impedisce il rendering e l'interazione.

SBAGLIATO: tutto sul main thread

```
Kotlin
// ERRORE: blocca il main thread!
fun onClick() {
    val url = "https://api.example.com/data"
    // Questa chiamata blocca per secondi
    val response = URL(url)
        .readText() // NetworkOnMainThread!
    binding.result.text = response
}
```

CORRETTO: lavoro in background

```
Kotlin
// OK: usa una coroutine
fun onClick() {
    lifecycleScope.launch {
        val response = withContext(
            Dispatchers.IO
        ) {
            URL("https://api.example.com/data")
                .readText()
        }
        // torna sul main per la UI
        binding.result.text = response
    }
}
```

Creare un Thread in Kotlin

Il modo più diretto per eseguire codice in background è creare un thread. Kotlin offre la funzione `thread {}`.

Modo 1: funzione `thread {}`

```
import kotlin.concurrent.thread

thread {
    // eseguito in background
    val data = loadFromNetwork()
    runOnUiThread {
        binding.text.text = data
    }
}
```

Kotlin

Modo 2: estendere `Thread`

```
class MyThread : Thread() {
    override fun run() {
        // lavoro in background
        val result = heavyComputation()
    }
}

// Avvio
MyThread().start()
```

Kotlin

Modo 3: `Runnable` (più flessibile)

```
val task = Runnable { Log.d("BG", "Eseguito in background") }
val t = Thread(task)
t.start()
t.join() // attende la fine (ATTENZIONE: blocca il chiamante!)
```

Kotlin

Problemi della gestione manuale dei Thread

Usare thread grezzi è semplice ma pieno di insidie che portano a bug difficili da riprodurre.

Race condition

Due thread che modificano la stessa variabile contemporaneamente producono risultati imprevedibili.

Memory leak

Un thread che tiene un riferimento a un'Activity impedisce al GC di raccoglierla dopo la rotazione.

CalledFromWrongThreadException

Toccare una View da un thread di background causa un crash immediato.

Difficoltà di cancellazione

Thread.stop() è deprecato. Interrompere un thread in modo pulito richiede flag manuali.

Nessuna struttura

Senza scope o gerarchia è facile perdere traccia dei thread attivi. Niente lifecycle awareness.

Scalabilità

Creare centinaia di thread consuma RAM (1 MB di stack ciascuno). Serve un pool.

Handler e Looper

Un Handler permette di inviare messaggi o Runnable alla coda del Looper di un thread (tipicamente il main thread).

```
// Creare un Handler legato al main thread
val handler = Handler(Looper.getMainLooper())

// Da un thread di background:
thread {
    val data = loadData() // lavoro pesante

    // Posta un Runnable sulla coda del main
    handler.post {
        binding.textView.text = data
    }

    // Posta con ritardo (2 secondi)
    handler.postDelayed({
        binding.textView.text = "Aggiornato!"
    }, 2000)
}
```

Kotlin

Flusso concettuale

1. Thread BG esegue il lavoro

2. handler.post { } accoda un Runnable

3. Looper del main thread lo preleva

4. Il Runnable aggiorna la UI

Kotlin Coroutines: concorrenza strutturata

Le coroutine sono il modo raccomandato per gestire il lavoro asincrono su Android. Leggere, cancellabili e lifecycle-aware.



Leggere

Migliaia di coroutine sullo stesso thread pool, senza sprecare 1 MB di stack ciascuna.



Cancellabili

La cancellazione si propaga da scope → figlie. Niente thread orfani.



Strutturate

Ogni coroutine vive in uno scope: lifecycleScope, viewModelScope, ecc.



Sequenziali

Il codice asincrono si legge dall'alto in basso, come fosse sincrono.

launch, Dispatchers e scope

launch avvia una coroutine "fire-and-forget". Il Dispatcher decide su quale thread pool gira.

```
// lifecycleScope = scope legato alla Activity/Fragment
lifecycleScope.launch {
    // Dispatchers.Main: main thread (default)
    binding.progress.isVisible = true

    // Dispatchers.IO: thread pool per I/O
    val data = withContext(Dispatchers.IO) {
        repository.fetchData()
    }

    // Dispatchers.Default: per calcoli CPU-bound
    val sorted = withContext(Dispatchers.Default) {
        data.sortedByDescending { it.score }
    }

    // Torniamo su Main automaticamente
    adapter.submitList(sorted)
    binding.progress.isVisible = false
}
```

Kotlin



withContext(dispatcher) { } sospende, non blocca: il thread resta libero per altre coroutine.

Funzioni suspend

Una funzione suspend può sospendersi senza bloccare il thread e riprendere quando il risultato è pronto.

```
// Dichiarare una funzione suspend
suspend fun fetchUser(id: Int): User {
    return withContext(Dispatchers.IO) {
        // chiamata di rete (sospende, non blocca)
        api.getUser(id)
    }
}

// Usarla in una coroutine
lifecycleScope.launch {
    try {
        val user = fetchUser(42)
        binding.name.text = user.name
    } catch (e: Exception) {
        binding.name.text = "Errore"
    }
}
```

Kotlin

Regole delle suspend fun

- Chiamabili solo da un'altra suspend fun o da una coroutine.
- Il compilatore le trasforma in una macchina a stati (continuation).
- Non creano un nuovo thread: sospendono quello corrente.
- Possono usare try/catch normalmente per gestire gli errori.
- Possono essere cancellate: controllano isActive internamente.

async / await: lavoro parallelo

async avvia una coroutine che restituisce un `Deferred<T>`. Con `await()` si ottiene il risultato.

```
lifecycleScope.launch {  
    // Avvio parallelo di due chiamate  
    val deferredUser = async(Dispatchers.IO) {  
        api.getUser(42)  
    }  
    val deferredOrders = async(Dispatchers.IO) {  
        api.getOrders(42)  
    }  
  
    // Attendo entrambi i risultati  
    val user = deferredUser.await()  
    val orders = deferredOrders.await()  
  
    // Aggiorno la UI (siamo su Main)  
    binding.name.text = user.name  
    binding.orderCount.text = "${orders.size} ordini"  
}
```

Kotlin



launch vs async

launch → Job (fire-and-forget). async → `Deferred<T>` (ritorna un valore). Entrambi sono cancellabili dallo scope.

viewModelScope e lifecycleScope

Android fornisce scope pre-configurati che si cancellano automaticamente al momento giusto.

lifecycleScope

Legato ad Activity / Fragment. Si cancella in onDestroy().

```
// In un Fragment
lifecycleScope.launch {
    val data = repository.load()
    binding.text.text = data
}
```

Kotlin

viewModelScope

Legato al ViewModel. Si cancella quando il VM è cleared.

```
// In un ViewModel
viewModelScope.launch {
    val data = repo.load()
    _state.value = data
}
```

Kotlin

Cos'è un Service

Un Service è un componente dell'applicazione (come Activity, BroadcastReceiver, ContentProvider) che esegue operazioni lunghe in background senza un'interfaccia utente.

- Non ha una UI: lavora "dietro le quinte".
- Gira sul MAIN THREAD di default (deve creare il suo thread se necessario!).
- Viene dichiarato nel AndroidManifest.xml.
- Sopravvive alla distruzione dell'Activity che lo ha avviato.
- Il sistema operativo può ucciderlo se ha bisogno di memoria.

 **ATTENZIONE:** un Service NON è un thread! Gira sul main thread a meno che non venga creato esplicitamente un thread o una coroutine.

Started

Avviato con `startService()`, vive finché non si ferma.

Bound

Legato a un componente con `bindService()`, vive finché qualcuno è connesso.

Foreground

Visibile tramite notifica, difficile da uccidere.

Ciclo di vita del Service

Il Service ha callback specifici che ne descrivono le fasi principali.

Started Service

onCreate()

creazione del Service

onStartCommand()

riceve l'Intent, inizia il lavoro

...lavoro in corso...**stopSelf() / stopService()**

richiesta di arresto

onDestroy()

rilascio risorse

Bound Service

onCreate()

creazione del Service

onBind()

ritorna un IBinder al client

...client connessi...**onUnbind()**

tutti i client disconnessi

onDestroy()

rilascio risorse

Started Service: esempio

Un Started Service viene avviato da `startService(intent)` e deve fermarsi da solo con `stopSelf()`.

```
class DownloadService : Service() {  
    private val scope = CoroutineScope(  
        Dispatchers.IO + SupervisorJob()  
    )  
    override fun onStartCommand(  
        intent: Intent?, flags: Int, startId: Int  
    ): Int {  
        val url = intent?.getStringExtra("url") ?: return START_NOT_STICKY  
        scope.launch {  
            downloadFile(url)  
            stopSelf(startId) // si ferma da solo  
        }  
        return START_NOT_STICKY  
    }  
    override fun onBind(intent: Intent?): IBinder? = null  
    override fun onDestroy() {  
        scope.cancel()  
        super.onDestroy()  
    }  
}
```

Kotlin

Dichiarare e avviare il Service

Ogni Service deve essere dichiarato nel Manifest. Si avvia con un Intent esplicito.

AndroidManifest.xml

```
<application ...>
  <service
    android:name=".DownloadService"
    android:exported="false" />
</application>
```

XML

Avvio dalla Activity

```
val intent = Intent(this,
  DownloadService::class.java)
intent.putExtra("url",
  "https://example.com/file.zip")
startService(intent)
```

Kotlin

Valori di ritorno di onStartCommand()

Flag	Comportamento
START_NOT_STICKY	Non ricrearlo se il sistema lo uccide. Ideale per task "usa e getta".
START_STICKY	Ricrealo con intent null. Ideale per servizi che restano in ascolto (es. music player).
START_REDELIVER_INTENT	Ricrealo ri-consegnando l'ultimo Intent. Ideale se il dato nell'intent è critico.

Bound Service: esempio con Binder

Un Bound Service permette a un componente (Activity, Fragment) di connettersi ad esso e chiamare i suoi metodi.

```
class TimerService : Service() {  
    private val binder = LocalBinder()  
    private var seconds = 0  
  
    inner class LocalBinder : Binder() {  
        fun getService(): TimerService = this@TimerService  
    }  
  
    override fun onBind(intent: Intent?): IBinder = binder  
  
    fun getElapsedSeconds(): Int = seconds  
  
    override fun onCreate() {  
        super.onCreate()  
        // avvia un contatore in background  
        CoroutineScope(Dispatchers.Default).launch {  
            while (true) { delay(1000); seconds++ }  
        }  
    }  
}
```

Kotlin

Connettersi a un Bound Service

L'Activity si connette con `bindService()` e riceve il `Binder` nella callback `onServiceConnected`.

```
class MainActivity : AppCompatActivity() {  
    private var timerService: TimerService? = null  
    private var bound = false  
  
    private val connection = object : ServiceConnection {  
        override fun onServiceConnected(name: ComponentName, binder: IBinder) {  
            timerService = (binder as TimerService.LocalBinder).getService()  
            bound = true  
        }  
        override fun onServiceDisconnected(name: ComponentName) {  
            bound = false  
        }  
    }  
    override fun onStart() {  
        super.onStart()  
        bindService(Intent(this, TimerService::class.java), connection, BIND_AUTO_CREATE)  
    }  
    override fun onStop() {  
        super.onStop()  
        if (bound) { unbindService(connection); bound = false }  
    }  
}
```

Kotlin

Foreground Service

Un Foreground Service mostra una notifica persistente all'utente. Il sistema lo considera visibile e quindi non lo uccide facilmente.



Riproduzione musicale



Download di file grandi



Tracciamento fitness /
GPS



Sincronizzazione dati

```
override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {  
    val notification = NotificationCompat.Builder(this, CHANNEL_ID)  
        .setContentTitle("Download in corso")  
        .setSmallIcon(R.drawable.ic_download)  
        .setProgress(100, 0, true)  
        .build()  
  
    startForeground(NOTIFICATION_ID, notification)  
    // avvia il lavoro in background con coroutine  
    scope.launch { performDownload() }  
    return START_NOT_STICKY  
}
```

Kotlin

Permessi per Foreground Service

Da Android 9+ servono permessi specifici nel Manifest. Da Android 14+ il tipo deve essere dichiarato.

```
<manifest ...>
  <!-- Permesso base (API 28+) -->
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>

  <!-- Tipo specifico (API 34+) -->
  <uses-permission android:name=
    "android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK"/>

  <application ...>
    <service
      android:name=".MusicService"
      android:exported="false"
      android:foregroundServiceType="mediaPlayback" />
    </application>
</manifest>
```

XML

Tipi di foreground service (API 34+)

camera • connectedDevice • dataSync • health • location • mediaPlayback • mediaProjection • microphone • phoneCall • remoteMessaging • shortService • specialUse

WorkManager: task differiti e garantiti

WorkManager è la soluzione Jetpack per lavoro in background che deve essere eseguito anche se l'app viene chiusa o il dispositivo riavviato.

Esecuzione garantita

Il lavoro viene completato anche dopo chiusura app o reboot.

Constraint

Esegui solo con Wi-Fi, batteria carica, storage disponibile.

Retry e backoff

Ritenta automaticamente con politica lineare o esponenziale.

Chaining

Combina worker in catene sequenziali o parallele.

Observability

Osserva lo stato con LiveData o Flow.

Periodic work

Ripeti ogni N minuti (minimo 15 min).

Creare un Worker

Un Worker è una classe che definisce il lavoro da eseguire. Il metodo `doWork()` gira su un thread di background.

```
class UploadWorker(  
    context: Context,  
    params: WorkerParameters  
) : CoroutineWorker(context, params) {  
  
    override suspend fun doWork(): Result {  
        return try {  
            val fileUri = inputData.getString("file_uri")  
            ?: return Result.failure()  
  
            // Lavoro effettivo (gira su Dispatchers.Default)  
            uploadFile(fileUri)  
  
            // Dati di output  
            val output = workDataOf("uploaded" to true)  
            Result.success(output)  
        } catch (e: Exception) {  
            if (runAttemptCount < 3) Result.retry()  
            else Result.failure()  
        }  
    }  
}
```

Kotlin

Schedulare ed osservare il lavoro

Dalla Activity/ViewModel si crea una WorkRequest, si impongono i vincoli e la si accoda.

```
// Constraint: solo con Wi-Fi e batteria non scarica
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.UNMETERED)
    .setRequiresBatteryNotLow(true)
    .build()

// Dati di input
val data = workDataOf("file_uri" to "/path/to/file")

// Richiesta one-time
val request = OneTimeWorkRequestBuilder<UploadWorker>()
    .setConstraints(constraints)
    .setInputData(data)
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS)
    .build()

// Accoda
WorkManager.getInstance(context).enqueue(request)

// Osserva lo stato
WorkManager.getInstance(context)
    .getWorkInfoByIdLiveData(request.id)
    .observe(this) { info -> /* info.state, info.outputData */ }
```

Kotlin

Lavoro periodico e catene

WorkManager supporta task periodici (minimo 15 min) e catene di worker.

Lavoro periodico

```
val periodic = PeriodicWorkRequestBuilder
    <SyncWorker>(
        1, TimeUnit.HOURS
    )
    .setConstraints(constraints)
    .build()

WorkManager.getInstance(context)
    .enqueueUniquePeriodicWork(
        "sync", ExistingPeriodicWorkPolicy.KEEP,
        periodic
    )
```

Kotlin

Catena sequenziale

```
WorkManager.getInstance(context)
    .beginWith(compressRequest)
    .then(uploadRequest)
    .then(cleanupRequest)
    .enqueue()

// beginWith puo accettare una lista
// per parallelismo:
// .beginWith(listOf(a, b))
// .then(merge)
// .enqueue()
```

Kotlin

Quando usare WorkManager vs Service

Scenario	Soluzione
Task differibile, deve sopravvivere al reboot	WorkManager
Operazione in tempo reale visibile all'utente	Foreground Service
Comunicazione client-server intra-processo	Bound Service

Confronto: Thread vs Coroutine vs Service

Ogni strumento ha il suo posto. Scegli in base a durata, visibilità e necessità di garanzia.

	Thread	Coroutine	Service	WorkManager
Peso	1 MB stack	~ KB	componente	componente
Cancellazione	manuale	scope-based	stopSelf()	automatica
Lifecycle-aware	no	sì (scope)	sì (binding)	sì (constraint)
Sopravvive all'app	no	no	sì	sì + reboot
Quando	mai da solo	I/O, calcoli	audio, GPS	sync, upload



Regola moderna: usa le Coroutine per il lavoro asincrono in-app, Service per operazioni visibili all'utente, WorkManager per task differiti.

Best practice

DA FARE

- Usa lifecycleScope / viewModelScope
- Dispatchers.IO per rete e disco
- Dispatchers.Default per calcoli CPU
- Foreground Service con notifica per audio/GPS
- WorkManager per sync periodica
- Cancella sempre le coroutine allo scope close
- Testa con StrictMode durante lo sviluppo

DA EVITARE

- Thread.sleep() sul main thread
- Thread grezzi senza scope o cancellazione
- Service senza thread/coroutine (blocca la UI!)
- GlobalScope (no lifecycle, memory leak)
- Lavoro lungo in BroadcastReceiver (max 10s)
- startService su Android 8+ in background
- Ignorare i constraint di WorkManager

Errori comuni e come risolverli

NetworkOnMainThreadException

Chiamata di rete sul main thread. Sposta su Dispatchers.IO.

CalledFromWrongThreadException

Modifica di una View da thread BG. Usa `withContext(Dispatchers.Main)`.

ANR dopo 5 secondi

Lavoro pesante sul main thread. Usa una coroutine o un Worker.

Service killed in background

Da API 26+ i bg service vengono uccisi. Usa Foreground o WorkManager.

Job cancellato senza motivo

Lo scope è stato distrutto (Activity chiusa). Usa `viewModelScope`.

Memory leak dall'Activity

Il thread tiene un riferimento all'Activity. Usa `scope lifecycle-aware`.

Cheat-sheet: esempi rapidi

I pattern più frequenti in un colpo d'occhio.

```
// 1. Coroutine semplice (Activity)
lifecycleScope.launch {
    val data = withContext(Dispatchers.IO) { api.fetch() }
    binding.text.text = data
}

// 2. Coroutine nel ViewModel
viewModelScope.launch {
    _state.value = repository.load()
}

// 3. Avviare un Foreground Service
val intent = Intent(this, MusicService::class.java)
ContextCompat.startForegroundService(this, intent)

// 4. WorkManager one-time
val req = OneTimeWorkRequestBuilder<SyncWorker>().build()
WorkManager.getInstance(this).enqueue(req)

// 5. Fermare un Service
stopService(Intent(this, DownloadService::class.java))
```

Kotlin

Albero decisionale: quale strumento usare?

Segui questo flusso per scegliere la soluzione giusta.

